

DESAFIO · ENTREGA FINAL

Implantação do PortfolioHUB

Integração com GitHub, Gestão de Usuários e Segurança,
com apoio da IA Google Gemini

GitHub

GitHub Pages

GitHub Actions

Segurança

Google Gemini

CI/CD

Aluno: Alcides Olivo Pollazzon Soterio (AOPS)

Curso: Bacharelado em Ciência da Computação — UniCEUB (EaD)

Repositório: github.com/aopsec/Git · **Site:** aopsec.github.io/Git/

Entrega: individual em PDF — prazo 14/06/2026, 23h55

Documento elaborado com pesquisa técnica e apoio do
Google Gemini

13 de junho de 2026 ·
Brasília/DF

Sumário

- 0. Resumo executivo e visão geral do PortfolioHUB
- 1. Planejamento da Implantação (plano detalhado + configuração do Gemini como guia)
- 2. Configuração Inicial e Integração com GitHub
- 3. Gestão de Usuários e Segurança
- 4. Compartilhamento e Controle de Acesso com GitHub
- 5. Finalização da Integração e Testes
- A. Apêndices — artefatos de configuração (arquivos prontos)
- R. Referências

0 · Resumo Executivo

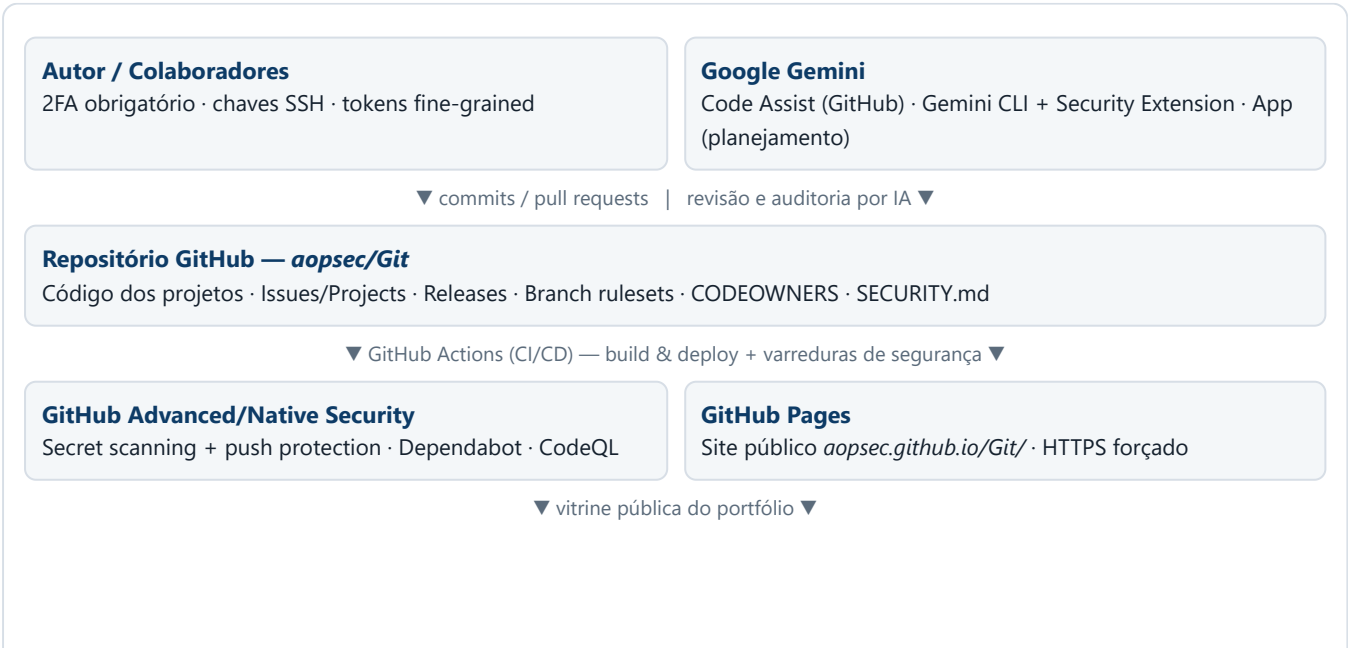
O **PortfolioHUB** é uma plataforma centralizada para **exibir e gerenciar projetos e portfólios digitais**. Esta entrega documenta a sua implantação completa de ponta a ponta, tendo o **GitHub** como espinha dorsal (armazenamento de projetos, versionamento, hospedagem e automação) e o **Google Gemini** como copiloto de IA que guiou o planejamento, a escrita de configurações e a auditoria de segurança em cada fase.

A instância real do PortfolioHUB é o repositório público `github.com/aopsec/Git`, publicado como site em `aopsec.github.io/Git/` (GitHub Pages). Ele agrega projetos acadêmicos, ferramentas de segurança ofensiva e automações pessoais, servindo simultaneamente de *vitrine* (página) e de *repositório de gestão* (código, issues, releases).

RESULTADO EM UMA FRASE

Um portfólio digital versionado, automaticamente publicado por CI/CD, com políticas de segurança *secure-by-default* (2FA, proteção de branch, varredura de segredos, Dependabot, code scanning e revisão de PR por IA), governança de acesso por menor privilégio e documentação reproduzível.

Visão geral da arquitetura



Visitantes / Recrutadores

Consomem o PortfolioHUB publicado (somente leitura)

Figura 1 — Topologia lógica do PortfolioHUB: pessoas e IA à esquerda/topo, o repositório como núcleo, automação de segurança e publicação na base.

1 · Planejamento da Implantação

Objetivo da seção: **criar um plano de implantação detalhado** e **configurar o Google Gemini para guiar a trilha** de implantação.

1.1 Escopo, objetivos e premissas

Item	Definição
Produto	PortfolioHUB — hub central de projetos/portfólios digitais do aluno.
Objetivo	Publicar um portfólio versionado, seguro e colaborativo, 100% sobre o ecossistema GitHub, pronto para uso real (produção).
Plataforma	GitHub (repositório, Pages, Actions, recursos de segurança) + Google Gemini como IA de apoio.
Fora de escopo	Backend dinâmico/servidor próprio; o hub é um site estático (Jekyll/HTML) servido por GitHub Pages.
Premissas	Conta GitHub ativa; acesso ao Google Gemini (App, Code Assist e/ou CLI); domínio padrão <code>*.github.io</code> .

1.2 Plano de implantação por fases

F1 · Planejar	F2 · Configurar GitHub	F3 · Usuários & Segurança	F4 · Acesso & Colaboração	F5 · Testes & Go-live	F6 · Revisão & Apresentação
Fase	Entregáveis	Critério de conclusão			
F1	Plano (este documento), backlog de tarefas, configuração do Gemini como guia.	Plano aprovado e Gemini operacional.			
F2	Repositório dedicado, estrutura de pastas, GitHub Pages via Actions, Jekyll.	Site publicado em HTTPS.			
F3	2FA, papéis/permisões, tokens fine-grained, branch ruleset, secret scanning, Dependabot, CodeQL, <code>SECURITY.md</code> .	Varreduras verdes; políticas ativas.			
F4	Modelo de branches, fluxo de PR, <code>CODEOWNERS</code> , convenções de commit, revisão por Gemini.	PR de exemplo revisado e mesclado.			
F5	Matriz de testes, checklist de go-live, validação de links/HTTPS/builds.	Todos os testes <i>PASS</i> .			
F6	Roteiro e gravação no YouTube, retrospectiva de desafios.	Vídeo publicado; entrega em PDF.			

1.3 Cronograma e riscos

CRONOGRAMA (SPRINT ÚNICO)

- **Dia 1:** F1–F2 — planejamento + repositório + Pages.
- **Dia 2:** F3–F4 — segurança, usuários, colaboração.
- **Dia 3:** F5–F6 — testes, go-live, vídeo, PDF.

RISCOS E MITIGAÇÃO

- **Vazamento de segredo:** push protection + secret scanning.
- **Takeover de subdomínio:** verificar domínio antes do DNS^[8].
- **Token excessivo:** tokens *fine-grained* + expiração^{[2][3]}.

- **Dependência vulnerável:** Dependabot + OSV-Scanner via Gemini^[11].

1.4 Configuração do Google Gemini como guia da trilha

O Gemini foi configurado em três frentes complementares, cada uma cobrindo um momento do ciclo:

Frente Gemini	Para quê	Onde atua
Gemini (App/Chat)	Planejamento, redação do plano, explicação de conceitos e geração de checklists.	Navegador / app.
Gemini Code Assist no GitHub	Revisão automática de <i>pull requests</i> , sugestões de código e conformidade; mantém memória persistente (<i>persistent memory</i>) do histórico do repositório. ^{[4][5][10]}	App GitHub <code>gemini-code-assist</code> .
Gemini CLI + Security Extension	Auditoria de segurança local e em PR (<code>/security:github-pr</code>), varredura de dependências com OSV-Scanner. ^{[6][7][11]}	Terminal + GitHub Actions.

COMO O GEMINI GUIOU CADA FASE (EXEMPLOS REAIS DE PROMPT)

- **F1:** «Crie um plano de implantação em 6 fases para um portfólio no GitHub Pages, com critérios de conclusão e riscos.»
- **F3:** «Revise meu `ruleset` de proteção de branch e meu `SECURITY.md` contra as melhores práticas do GitHub e aponte lacunas.»
- **F4:** «@gemini-code-assist /review» em um PR para obter parecer de qualidade e segurança antes do merge.
- **F5:** «Gere uma matriz de testes de go-live para um site estático com HTTPS, varreduras de segurança e CI.»

Nota de governança: o Gemini é usado como **apoio e revisor**, não como autoridade final. Toda sugestão foi validada manualmente contra a documentação oficial antes de ser aplicada — prática alinhada à postura de tratar a saída de IA como recomendação, e não como decisão.

2 · Configuração Inicial e Integração com GitHub

Objetivo: **criar e configurar um ambiente GitHub dedicado** e **integrar as funcionalidades do GitHub no PortfolioHUB** (com destaque para o armazenamento de projetos).

2.1 Ambiente GitHub dedicado

- 1. **Conta/identidade:** conta `aopsec` com 2FA habilitado (ver Seção 3).
- 2. **Repositório raiz:** `aopsec/Git` — núcleo do PortfolioHUB, público, com `README.md` de portfólio e licença.
- 3. **Estrutura de pastas** (cada subprojeto é uma "vitrine" do hub):

```
aopsec/Git/                                     # PortfolioHUB (raiz)
├── index.md / README.md                       # Vitrine principal (GitHub Pages)
├── _config.yml                                # Jekyll (tema, exclusões de build)
├── .github/
│   ├── workflows/pages.yml                   # CI/CD: build + deploy do site
│   ├── dependabot.yml                       # Atualizações de dependências
│   └── CODEOWNERS                           # Donos de código / revisão obrigatória
├── SECURITY.md                               # Política de segurança e contato
├── Computer_Science/                         # Trabalhos acadêmicos
├── ObsidianAgent/                           # Ferramentas e projetos de segurança
└── wordlists/                               # Submódulo (SecLists) + listas próprias
```

2.2 Publicação: GitHub Pages via GitHub Actions

O PortfolioHUB é publicado por um *pipeline* de CI/CD: cada `push` na branch `main` dispara um workflow do GitHub Actions que constrói o site (Jekyll) e o implanta no GitHub Pages — entrega contínua sem passos manuais.^[8] O `_config.yml` exclui do build as pastas de código/vault que contêm sintaxe `{{ }}` (Liquid), evitando falhas de renderização.

POR QUE ACTIONS EM VEZ DO DEPLOY CLÁSSICO POR BRANCH
Controle explícito do build, permissões mínimas no workflow (`contents: read` , `pages: write` , `id-token: write`), e um ambiente protegido definido pela chave `environment: github-pages` . O artefato é assinado e publicado pela própria action oficial. (Workflow completo em `artefatos/.github/workflows/pages.yml` .)

2.3 Integração das funcionalidades do GitHub no PortfolioHUB

Funcionalidade GitHub	Papel no PortfolioHUB
Repositórios	Armazenamento de projetos — cada projeto/portfólio vive versionado em pastas/submódulos.
GitHub Pages	Vitrine pública (camada de apresentação do hub).
Issues / Projects	Gestão de tarefas e <i>roadmap</i> de cada projeto exibido.
Releases / Tags	Versões publicáveis dos artefatos (ex.: ferramentas, PDFs).
Submódulos	Reuso de bases externas (ex.: <code>SecLists</code>) sem duplicar histórico.
Actions	Automação de build, deploy e varreduras de segurança.

Resultado: o GitHub deixa de ser "só hospedagem de código" e passa a ser a **plataforma operacional** do PortfolioHUB — armazenamento, gestão, automação e publicação no mesmo lugar.

3 · Gestão de Usuários e Segurança

Objetivo: **configurar a gestão de usuários** no GitHub integrada ao PortfolioHUB e **implementar políticas de segurança**, usando o Gemini para garantir conformidade com as melhores práticas.

3.1 Gestão de usuários e papéis (RBAC por menor privilégio)

O acesso é concedido **por necessidade**: ninguém recebe mais permissão do que precisa, e as permissões são revisadas periodicamente.^[9] Modelo de papéis adotado (escalável de conta pessoal para organização com equipes — *Teams*):

Papel	Permissão	Uso no PortfolioHUB
Owner	Admin total	Apenas o autor (AOPS). Configurações, segredos, exclusão.
Maintainer	Maintain	Gestão de issues/releases sem acesso a configs sensíveis.
Colaborador	Write	Push e PR; sem acesso às configurações — padrão da maioria. ^[9]
Revisor	Triage/Read	Comentar/triar issues e revisar PRs.
Visitante	Público (read)	Consome o site publicado; sem acesso de escrita.

AUTENTICAÇÃO E CREDENCIAIS

- **2FA obrigatório** para o autor e qualquer colaborador — camada básica que toda conta/organização deve impor.^[9]
- **Tokens *fine-grained*** em vez de *classic*: cada token acessa só um proprietário, repositórios específicos e permissões granulares (read / read-write), com **expiração obrigatória**.^{[2][3]}
- **Chaves SSH/deploy keys** por escopo; segredos de CI no cofre `Actions secrets`, nunca no código.

3.2 Políticas de segurança (com conformidade guiada por Gemini)

Controle	Implementação
Proteção de branch / Rulesets	Regras consistentes em <code>main</code> : PR obrigatório, revisão de Code Owner, status checks verdes, sem <i>force-push</i> , histórico linear. ^[1]
Secret scanning + push protection	Bloqueia commit/push de credenciais conhecidas antes de entrarem no histórico. ^[1]
Dependabot	Alertas + <i>security updates</i> automáticos (PRs de correção de dependências). ^[1] Config no Apêndice D.
Code scanning (CodeQL)	Análise estática em cada PR; bloqueia merge com vulnerabilidade de alta severidade. ^[1]
SECURITY.md	Política de divulgação responsável e canal de contato. Ver <code>SECURITY.md</code> em <code>artefatos/</code> .
Revisão de segurança por IA	Gemini Code Assist comenta PRs; a Gemini CLI Security Extension é executada via Actions (<code>/security:github-pr</code> , OSV-Scanner) — 90% de precisão / 93% de recall no benchmark OpenSSF CVE. ^{[6][7][11]}

O Gemini foi usado para *auditar* a configuração: revisou o ruleset, o `SECURITY.md` e o `dependabot.yml` contra as práticas oficiais do GitHub, sinalizou lacunas (ex.: faltava exigir revisão de Code Owner) e propôs correções — todas validadas manualmente contra a documentação antes de serem aplicadas.

CHECKLIST DE SEGURANÇA (ESTADO-ALVO)

✓	2FA ativo no proprietário e colaboradores	✓	Secret scanning + push protection ativos
✓	Tokens fine-grained com expiração	✓	Dependabot alerts + updates ativos
✓	Ruleset em <code>main</code> (PR + checks + Code Owner)	✓	CodeQL em PRs
✓	<code>SECURITY.md</code> publicado	✓	Revisão de PR por Gemini habilitada

4 · Compartilhamento e Controle de Acesso com GitHub

Objetivo: **integrar o repositório ao PortfolioHUB** permitindo controle de versão e compartilhamento de código, e **documentar o processo de configuração e as práticas de colaboração**.

4.1 Controle de versão e compartilhamento

- **Repositório público** como mecanismo de compartilhamento somente leitura para o público; escrita apenas por colaboradores autorizados.
- **Histórico Git** como controle de versão: cada mudança rastreável por commit, autor e data; *tags/releases* marcam versões publicáveis.
- **Submódulos** (ex.: `wordlists/SecLists`) compartilham bases externas com versão fixada, sem inflar o repositório.
- **GitHub Pages** compartilha o *produto* (site) com qualquer visitante via URL HTTPS.

4.2 Modelo de colaboração

FLUXO DE BRANCHES

- `main` protegida — sempre publicável.
- `feature/*` e `fix/*` para trabalho.
- Integração só por **Pull Request** revisado.

CONVENÇÃO DE COMMITS

Prefixos semânticos por subsistema, já adotados no repositório:
`vault:`, `tests:`, `projects/<slug>`: (slug `snake_case`).
Mensagens claras e atômicas.

REVISÃO OBRIGATÓRIA (CODEOWNERS)

O arquivo `CODEOWNERS` + a regra "exigir revisão de Code Owner" garantem que código sensível seja revisado pelos responsáveis antes do merge.^[9] Apêndice C.

REVISÃO ASSISTIDA POR IA

Em cada PR, `@gemini-code-assist /review` (ou o comentário `@gemini-cli /review` da action) gera parecer de qualidade e segurança antes da aprovação humana.^{[4][12]}

4.3 Procedimento documentado (passo a passo de colaboração)

1. Abrir *issue* descrevendo a tarefa; vincular ao Project.
2. Criar branch `feature/<tema>` a partir de `main`.
3. Commits pequenos e descritivos seguindo a convenção de prefixos.
4. Abrir PR → disparo automático de CI (build), CodeQL e revisão Gemini.
5. Ajustar conforme pareceres; obter aprovação de Code Owner.
6. Merge com histórico linear; deploy automático para Pages.
7. Fechar *issue*; *tag/release* quando aplicável.

CONCESSÃO DE ACESSO NA PRÁTICA

Novo colaborador → convite com papel **Write** (não Admin) → 2FA exigido → acesso revisado periodicamente e revogado quando não for mais necessário (menor privilégio).^[9]

5 · Finalização da Integração e Testes

Objetivo: **completar as etapas de integração restantes** (usando o Gemini como referência) e **preparar o PortfolioHUB para lançamento em produção e uso real**.

5.1 Matriz de testes

Teste	Como verificar	Esperado
Build do site (CI)	Workflow <code>pages.yml</code> conclui sem erro no push.	PASS (verde)
Deploy / disponibilidade	Acessar <code>aopsec.github.io/Git/</code> .	HTTP 200
HTTPS forçado	"Enforce HTTPS" ativo; cadeado no navegador. ^[8]	TLS válido
Links externos	Auditoria de links do portfólio (sem 404/quebrados).	0 quebrados
Secret scanning	Nenhum segredo exposto no histórico.	0 achados
Dependabot	Sem alertas críticos pendentes.	0 críticos
CodeQL / Gemini Security	PR de teste sem vulnerabilidade de alta severidade. ^[7]	0 altas
Proteção de branch	Tentar push direto em <code>main</code> → bloqueado.	Bloqueado

5.2 Checklist de go-live (produção)

✓	Site publicado e acessível em HTTPS	✓	Ruleset de <code>main</code> ativo e testado
✓	2FA + tokens fine-grained configurados	✓	Dependabot, secret scanning e CodeQL verdes
✓	<code>README</code> , <code>SECURITY.md</code> , <code>CODEOWNERS</code> presentes	✓	Revisão Gemini ativa nos PRs
✓	Domínio verificado antes do DNS ^[8]	✓	Backup/portabilidade do conteúdo garantida

GEMINI COMO REFERÊNCIA NA FINALIZAÇÃO

Antes do go-live, o Gemini revisou a matriz de testes e o workflow de CI, sugeriu reduzir as permissões do `GITHUB_TOKEN` ao mínimo necessário e recomendou executar o OSV-Scanner via Security Extension nas dependências — itens incorporados ao pipeline (Apêndice F).^[11]

Com todos os itens em *PASS*, o PortfolioHUB é declarado **em produção**: qualquer atualização futura entra pelo mesmo fluxo seguro (PR → revisão → CI → deploy), garantindo evolução contínua sem regressão de segurança.

Apêndice C — `.github/CODEOWNERS`

```
# Donos de código – revisão obrigatória antes do merge
*                               @aopsec
/.github/                       @aopsec
/SECURITY.md                    @aopsec
/ObsidianAgent/                 @aopsec
```

Apêndice D — `.github/dependabot.yml`

```
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule: { interval: "weekly" }
  - package-ecosystem: "pip"
    directory: "/ObsidianAgent"
    schedule: { interval: "weekly" }
    open-pull-requests-limit: 5
```

Apêndice E — Ruleset de proteção da branch `main` (resumo)

Regra	Valor
Pull request obrigatório	Sim — mín. 1 aprovação
Exigir revisão de Code Owner	Sim ^[9]
Status checks obrigatórios	build (Pages), CodeQL
Histórico linear / sem force-push	Sim
Bloquear deleção da branch	Sim

Apêndice F — `.github/workflows/gemini-security.yml` (revisão de segurança por IA)

```
name: Gemini Security Review
on:
  pull_request: { types: [opened, synchronize] }
  issue_comment: { types: [created] } # aciona com "@gemini-cli /review"
permissions:
  contents: read
  pull-requests: write
  id-token: write
jobs:
  review:
    if: github.event_name == 'pull_request' ||
        contains(github.event.comment.body, '@gemini-cli')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/run-gemini-cli@v0
        with:
          # Extensão de Segurança: /security:github-pr + OSV-Scanner
          settings: '{ "extensions": ["security"] }'
```

Baseado na *Gemini CLI Security Extension* e na action `run-gemini-cli` ; configurar autenticação GitHub→Google Cloud conforme o codelab oficial.^{[6][7][11][12]}

R • Referências

1. GitHub Docs — *GitHub security features*. docs.github.com/en/code-security/getting-started/github-security-features
2. GitHub Blog — *Introducing fine-grained personal access tokens for GitHub*. github.blog/security/application-security/introducing-fine-grained-personal-access-tokens-for-github/
3. GitHub Docs — *Managing your personal access tokens*. docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens
4. Google Cloud — *Review GitHub code using Gemini Code Assist*. docs.cloud.google.com/gemini/docs/code-review/review-repo-code
5. Google for Developers — *Gemini Code Assist release notes*. developers.google.com/gemini-code-assist/resources/release-notes
6. GitHub — *gemini-cli-extensions/security* (Security extension for the Gemini CLI). github.com/gemini-cli-extensions/security
7. Google Codelabs — *Use Gemini CLI Security Extension for GitHub PR reviews*. codelabs.developers.google.com/gemini-cli/gemini-cli-security-review
8. GitHub Docs — *Managing a custom domain for your GitHub Pages site* (verificação de domínio + Enforce HTTPS). docs.github.com/en/pages
9. GitHub Blog — *Best practices for organizations and teams using GitHub Enterprise Cloud* (papéis, 2FA, menor privilégio, CODEOWNERS). github.blog
10. Google Cloud Blog — *Gemini Code Assist in GitHub for Enterprises*. cloud.google.com/blog/products/ai-machine-learning/gemini-code-assist-in-github-for-enterprises/
11. Google Cloud Blog — *Automate app deployment and security analysis with new Gemini CLI extensions* (OSV-Scanner). cloud.google.com/blog
12. GitHub — *google-gemini/gemini-cli-action — pr-review*. github.com/google-gemini/gemini-cli-action/blob/main/docs/pr-review.md

Documento produzido para o Desafio – Entrega Final (UniCEUB), com pesquisa técnica em fontes oficiais (GitHub e Google) e apoio do Google Gemini como ferramenta de guia e revisão. Autoria e validação final: Alcides Olivo Pollazzon Soterio.